MASTARA

Consolidated Product & Technical Requirements

Student Registration • Bundle Size & Pricing • Extra Single Courses • Access Duration • Offline-First Content

Specification basis

Prepared from the supplied Mastara SRS v3.2, “SRS Update Sections (after v3.2)” dated 27 October 2026, and the implementation refinements and phase roadmap supplied in this conversation.

Purpose

Resolve pricing and bundle-count ambiguity and provide one implementation-ready requirements document for Phase 1 onward. This document preserves the agreed offline-first, deferred-registration and manual-receipt approval approach while adding explicit formulas, validation rules and acceptance criteria.

Version: 1.0 — Consolidated proposal for product-owner approval

Date: 28 October 2026

1. Executive Summary

Mastara shall support local-only student registration, server registration only after a qualifying bundle payment is approved, admin-configurable bundle size limits and prices, count-based pricing for extra single courses, optional downloads, protected offline access, and automatic entitlement expiry.

The central rule is: the server is authoritative for eligibility, course counts, prices, payment approval, student IDs, entitlements and expiry. Mobile UI restrictions improve clarity but are never the only enforcement.

1.1 Decisions consolidated in this document

- **First purchase:** batch, semester or full-course bundle only. A single-course order cannot create a student account.
- **Manual receipt approval:** treat this as the launch workflow, consistent with SRS v3.2. The MS number is assigned only when Operations Admin approves the qualifying bundle receipt.
- **Bundle sizes:** Super Admin configures minimum and maximum course counts separately for semester and full-course products. Batch bundle limits are separately configurable rather than assumed to share the same limits.
- **Bundle price:** default calculation is based on the count and admin-defined prices of included courses, with an optional configured bundle discount. Super Admin may instead set a fixed bundle price override.
- **Extra single courses:** allowed only while a qualifying bundle is active and single-course sales are enabled. Price is calculated from the configured tier schedule and the count of extra courses in the order.
- **Access duration:** each product has an admin-configured duration in days. The entitlement starts when payment is approved, not when the student first opens content.
- **Refunds:** refunding an order revokes entitlements created by that order. Separately purchased courses are not revoked merely because a different bundle expires; refunding the single-course order itself revokes that course entitlement.

1.2 Normative language

MUST is mandatory, **SHOULD** is recommended, and **MAY** is optional. If this document is approved, its commerce rules should replace conflicting older pricing rules in SRS v3.2 and be merged into the next consolidated SRS version.

2. Scope and Architecture

Applies to the React Native/Expo student app, NestJS REST API, PostgreSQL database, Next.js Admin Panel, shared domain/contracts packages, local SQLite and secure storage, and S3-compatible object storage.

Area	Authoritative requirement
Mobile app	Local registration/profile, store UI, order submission, claim polling, SQLite migration, streaming, downloads and offline entitlement checks.
API	Eligibility, server-side price calculation, receipt/order validation, approval, claim-secret validation, MS assignment, device binding, entitlements and expiry.
Database	Transactional order/payment/entitlement records, pricing configuration, count limits, course versions and unique student identity.
Admin Panel	Paid-only student directory, receipt approval queue, bundle limits, course prices, bundle pricing rules, extra-course tiers, durations and single-course toggle.
Domain/contracts	Shared DTOs and pure eligibility/pricing/access functions with unit tests.

3. Student Lifecycle and Registration

3.1 Before the first qualifying purchase — local only

- The student can register, sign in, and edit their profile offline. The app creates a random UUID localAccountId.
- No student profile, localAccountId, password, profile edits or learning progress is sent to the central server before the student submits a qualifying bundle order.
- The app stores only a salted, iterated password hash locally. At first purchase, the student re-enters the password; the password is sent only over HTTPS/TLS and is hashed server-side with bcrypt or an approved password-hashing algorithm.
- Show the notice: “Your account is saved on this phone until you buy a course.”
- Unpaid local accounts are not shown in the Admin Students directory. Losing/reinstalling the app before payment may lose the local account and unsynced progress.
- Catalog metadata/listings may sync without sending student registration data, consistent with the offline catalog requirement.

3.2 Order submission and pending state

- The first order MUST be a valid batch, semester or full-course bundle. The API rejects single-course orders from local/unregistered students.
- The app generates a cryptographically secure 32-byte (256-bit) claimSecret, stores it securely on the device, and sends only claimSecretHash = SHA-256(claimSecret) with the order.
- The order includes localAccountId, profile fields, re-entered password over TLS, selected product/bundle and course selection, receipt metadata, and claimSecretHash.
- The server stores pending profile fields and a password hash, not plaintext password. The order is marked pending and the student remains local-only with “Payment pending.”
- The public order endpoint MUST apply rate limits, strict image validation (JPEG/PNG/WebP), a configured maximum receipt size, and no more than two pending orders per phone number. Apply equivalent duplicate/order controls to email where present.

3.3 Approval and claim

1. Operations Admin reviews the receipt and approves or rejects it.
2. On approval of a valid first bundle, one database transaction creates the Student, allocates the next MS number, creates the device binding, marks the order approved, and creates entitlement records with starts_at and expires_at.
3. The student number is allocated by a transactional counter row, e.g. UPDATE counters SET value = value + 1 WHERE name = 'student_ms_code' RETURNING value, in the same transaction as student creation. A failed transaction rolls back the counter increment.
4. The app polls for readiness or receives a notification, then calls POST /orders/:orderId/claim with the original claimSecret.
5. The API verifies SHA-256(claimSecret) equals the stored claim_secret_hash, then returns studentCode, access/refresh tokens and entitlements. The claim operation is idempotent: repeated valid claims return the same account identity and do not create another student.
6. The app atomically migrates local SQLite rows and SecureStore references from localAccountId to the server student identity, including progress, attempts and download mappings. If migration fails, it retries safely and does not duplicate or lose records.

3.4 Existing account and takeover protection

If the submitted phone number or email already belongs to a paid student, the server MUST NOT silently link the order to that account. Require authentication to the existing account or a verified OTP flow. Otherwise return ACCOUNT_EXISTS_LOGIN_REQUIRED. Linking must not create a new MS number.

4. Product Catalogue, Bundle Size and Pricing Rules

4.1 Admin-controlled configuration

Super Admin controls product availability, price configuration, minimum/maximum course counts, bundle discount or fixed price, extra-course tier schedule, single-course enablement, and access duration. Changes are audited with actor, timestamp, previous value, new value and reason. Academic Admin cannot change commercial settings; Operations Admin may view them as permitted by the role matrix.

Setting	Required fields / validation
Semester bundle	Enabled; minCourses; maxCourses; pricingMode (CALCULATED or FIXED); optional discountPercent; optional fixedPriceETB; durationDays; allowed semester/stream scope.
Full-course bundle	Enabled; minCourses; maxCourses; pricingMode; optional discountPercent; optional fixedPriceETB; durationDays; included course scope and whether Common/COC/exams are included.
Batch bundle	Enabled; its own minCourses/maxCourses and pricing configuration. Do not silently inherit semester limits.
Course price	Admin-defined price per course, in ETB, with active/inactive state. The API snapshots price at order creation.
Extra-course tiers	Ordered count ranges and per-course price or discount for each range. Ranges must not overlap and must cover every allowed count.
Single-course sales	Global singleCoursesEnabled toggle. When off, new single-course orders are rejected even if the student has an active bundle.
Duration	Positive integer durationDays for each product type; changes affect new approvals only unless a separately audited entitlement adjustment is explicitly performed.

4.2 Bundle size validation

- For a semester bundle, all selected courses must belong to the same configured semester and stream unless the product definition explicitly permits otherwise.
- The number of unique eligible courses in the order MUST be between the configured minimum and maximum, inclusive.
- For a full-course bundle, eligible courses are determined by the product's configured scope (for example, all relevant stream courses, semesters and Common courses). The selected/covered count must satisfy the configured minimum and maximum where the product uses count limits.
- Duplicate course IDs are rejected or deduplicated before price calculation; they must never increase the count or charge twice.
- Both mobile and API validate the count. The API is authoritative and returns a clear error code and the accepted min/max values if the order is invalid.
- The UI should show selected count, minimum, maximum, included courses, price breakdown and any discount before receipt submission.

4.3 Bundle price formula

Default recommended mode: calculate the bundle amount from the current server-side prices of all eligible included courses, then apply the configured bundle discount. A Super Admin can choose a fixed bundle price override for products that must have a fixed advertised price.

Calculated mode:

```
BaseAmount = sum(coursePriceETB for each unique course covered by the bundle)
DiscountAmount = roundCurrency(BaseAmount × discountPercent / 100)
BundlePrice = max(0, BaseAmount − DiscountAmount)
```

Fixed mode:

BundlePrice = configured fixedPriceETB, provided the selected bundle satisfies all product scope and count rules. The UI may show the underlying course-price comparison, but the amount due is the fixed configured price.

Currency is ETB. Use integer minor units (for example, cents where supported) or a consistent decimal-money representation; never use floating-point arithmetic for authoritative money calculations. The API returns a server-calculated quote and the order stores a price snapshot. Client-submitted totals are ignored.

4.4 Extra single-course pricing based on count

An extra single course is a course outside the student's active bundle coverage. It can be bought only when the student has at least one active qualifying bundle and single-course sales are enabled. The API MUST reject courses already covered by an active entitlement as duplicates.

Recommended tier model: the Super Admin configures a price for each extra-course count band. The count is the ordinal position of each extra course within the same order, so the price preview is deterministic and easy to explain. The first extra course can cost more, with a lower price for subsequent extras.

Example tier (illustrative)	Price per course	How it is applied
Extra course #1	100 ETB	First unique extra course in this order.
Extra courses #2–#5	50 ETB each	Each extra course whose order position falls in this band.
Extra courses #6 and above	40 ETB each	Each additional course in this band.

Example: an order contains 7 eligible extra courses. Total = 1×100 + 4×50 + 2×40 = 380 ETB. These amounts are examples only; Super Admin sets the real tiers and prices.

The server computes the total from the course count and tier schedule, stores the per-course price breakdown on the order, and rejects a client total that differs. The app displays each course, tier position, unit price, subtotal and final total before receipt upload.

5. Purchase Eligibility and Entitlement Rules

Student / product state	Required behaviour
Local-only student; first purchase	Can submit only a valid batch, semester or full-course bundle order. A single-course order is rejected.
Bundle receipt pending	No MS number or paid entitlement yet. Student remains local-only and sees Payment pending.
Bundle approved	Create server student and MS number once; activate bundle entitlement; bind device; enable normal sync.
Active bundle; single-course sales on	May buy eligible courses not already covered by a bundle or other active entitlement.
Active bundle; single-course sales off	Single-course purchase is unavailable; API rejects it. Existing entitlements remain valid until expiry/revocation.
No active bundle	Cannot buy a new single course. Show locked button and message directing student to a bundle.
Course already included	Show Included; no Buy button; server rejects duplicate purchase.
Bundle expires	Bundle-provided access ends. Separately purchased course access continues until its own entitlement expires, unless that specific purchase is refunded/revoked.
Bundle refunded	Revoke entitlements created by that bundle order. Do not revoke independent single-course orders merely because they are independent.
Single-course order refunded	Revoke the entitlement created by that specific single-course order.

5.1 Which bundles unlock single purchases

Recommended resolution: batch, semester and full-course bundles all count as qualifying active bundles unless Super Admin explicitly disables single-course eligibility for a particular bundle type. The rule is configured and checked server-side. The student-facing message should say “Buy a qualifying bundle first to unlock single courses” if the eligible bundle types are configurable.

5.2 Price and eligibility snapshots

- The API creates a quote using current active settings and selected course count. The quote has an expiry and an opaque quote/order ID.
- At order creation, store the quoted unit prices, discount, count, total, product configuration version and currency.
- Later admin price changes apply to new quotes/orders only. Pending orders should either preserve their valid unexpired quote or require a refreshed quote after expiry.
- Approval validates that the order is still eligible and the receipt/order amount matches the stored expected amount. An admin must not be able to approve an invalid count or underpriced order by bypassing UI validation.
- No course is charged twice when covered by the same active bundle or active entitlement. Renewal behaviour must follow the product's renewal-window rule.

6. Access Duration, Expiry and Refunds

6.1 Entitlement lifecycle

- Every product has durationDays configured by Super Admin.
- On manual approval, entitlement starts_at is the approval timestamp and expires_at = starts_at + durationDays.
- Duration starts on approval, not order submission, receipt upload, claim, first app launch or first content open.
- Each separately purchased extra course has its own entitlement and expiry based on that single-course order's approval timestamp.

- Server time is authoritative. Device clock changes must not extend access. Offline entitlement tokens are server-signed, device-bound and expiry-limited, and the app retains last-seen server time to detect clock rollback.
- When online, entitlement renewal/revocation is synchronized. Offline content stops opening when the locally verifiable entitlement expires or is revoked as represented by the latest valid signed entitlement state.

6.2 Access decision

Use a shared domain rule, conceptually canOpen(student, item): first reject inactive/unpublished/deleted content; then allow if the student has an active entitlement for the item's course; then allow course-level free access; then allow item-level free access; otherwise lock. Content must also have an available source: online stream or a previously downloaded local copy.

Student state	Online	Offline
Local-only / no purchase	Free content may open online.	Free content opens only if previously downloaded.
Valid entitlement	Permitted content opens by streaming; download is optional.	Permitted content opens only if downloaded and the signed entitlement remains valid.
No entitlement for paid content	Locked Content/paywall.	Locked Content/paywall; downloaded file must not bypass entitlement.
Entitlement expired/revoked	API denies access and app updates local state.	Downloaded paid content becomes unreadable when the signed entitlement expires/revocation state is effective.

6.3 Refund policy

Use the existing request → approval/rejection → payout-recorded → entitlement-revoked workflow. Operations Admin requests a refund with a reason; Super Admin approves or rejects. Approval of a refund revokes entitlements tied to the refunded order and is audited. Refund payout is manual and out of scope for automation.

7. Claim Security, Idempotency and Student ID Allocation

7.1 Claim secret

- Generate 32 cryptographically random bytes for claimSecret; encode using a URL-safe representation.
- Transmit only SHA-256(claimSecret) in the order. Store claim_secret_hash server-side; never log or expose the plaintext secret.
- Claim endpoint accepts claimSecret only over HTTPS, verifies the hash using a constant-time comparison where practical, and rejects invalid, expired, or unrelated claims.
- Claim success is idempotent for the same approved order and local account. Repeated calls return the same student identity and a valid token/session response without creating a second account.
- Rate-limit claim attempts and audit suspicious failures. Do not expose whether an arbitrary order ID exists to unauthenticated callers beyond the minimum required response.

7.2 Gapless MS number

MS numbers start at MS0001 and are assigned by the server only when the first qualifying bundle payment is approved. Admin accounts never consume student numbers. Student creation, counter increment, order approval, device binding and initial entitlement creation occur in one transaction.

The counter must be updated transactionally using a locked row or an equivalent PostgreSQL mechanism that rolls back on transaction failure. A plain PostgreSQL sequence does not guarantee gapless numbers after rollbacks; do not use a non-transactional sequence if strict no-gaps is the requirement.

7.3 Duplicate local accounts

Use localAccountId as the idempotency key for the first qualifying order. A retry must return the same pending order/claim result and must not create a duplicate student or order. Phone/email collisions require login or verified OTP; never blindly merge local data into a paid account.

8. API, DTO and Data Model Requirements

8.1 Core contracts

Contract	Required fields / behaviour
SubmitOrderWithLocalDto	localAccountId, profile fields, password over TLS, claimSecretHash, productId/bundleType, unique course IDs or product scope, receipt payload/reference and quote/order token.
ClaimOrderDto	claimSecret; only accepted for an approved order belonging to the claim flow.
ClaimOrderResponseDto	studentCode, access token, refresh token, entitlements and any required migration/session metadata.
PaymentSettingsDto	bundle pricing mode, bundle prices/discounts, min/max course counts, course prices, extra-course tier schedule, singleCoursesEnabled, durationDays and config version.
Price quote	quoteId, product, unique course count, per-course breakdown, base amount, discount, total ETB, settings version, expiresAt.
Order result	orderId, status, amount snapshot, payment pending/approved/rejected state, claim readiness and safe error codes.

8.2 Data model changes

- Order.student_id nullable for pending unregistered orders.
- Order fields: local_account_id, claim_secret_hash, pending profile fields, pending_password_hash, quote/config snapshot, order type, selected course IDs or order items, expected amount and status timestamps.
- Student.local_account_id unique as an idempotency key where applicable; Student.student_code unique.

- Course.content_version integer default 1; bump when content is published/edited/deactivated or access status changes according to catalog-version policy.
- Entitlement stores product/order linkage, covered course, starts_at, expires_at, status and revocation metadata.
- Pricing configuration stores bundle min/max, pricing mode, fixed price or discount, course prices, extra-course tiers, duration and active configuration version.
- Order items or equivalent normalized records store each included/extra course and the applied unit price/tier to make the total auditable.
- Counter row for student_ms_code is updated in the same transaction that creates the student.

8.3 API enforcement

- Server calculates all amounts; client-provided price totals are never authoritative.
- Server checks bundle type, count range, course scope, duplicates, single-course eligibility, active bundle status, existing course entitlements and singleCoursesEnabled.
- Server validates receipt MIME by content where possible, size, image decoding and allowed formats; strip EXIF metadata and store receipts privately with signed access URLs.
- Apply route-specific rate limits, pending-order caps and audit events.
- Return stable error codes, including ACCOUNT_EXISTS_LOGIN_REQUIRED, BUNDLE_SIZE_INVALID, SINGLE_COURSE_BUNDLE_REQUIRED, SINGLE_COURSES_DISABLED, COURSE_ALREADY_INCLUDED, PRICE_QUOTE_EXPIRED, RECEIPT_INVALID and CLAIM_SECRET_INVALID.

9. Mobile App Requirements

9.1 Store and pricing UI

- Show bundle products with configured minimum/maximum course counts and allowed scope.
- As the student selects courses, show live count feedback: selected count, remaining slots, minimum needed and whether the bundle is valid.
- Display a server-generated price quote and itemized breakdown. Refresh the quote when course selection, product, or pricing configuration changes.
- Before any order submission, clearly show course list, count, base price, discount, total ETB, duration, and payment/receipt instructions.
- For a student without an active bundle, show single courses but lock the Buy action with: “Buy a semester or full bundle first to unlock single courses” (or the configured qualifying-bundle wording).
- For a course already covered by a bundle, show Included and no Buy action.
- For active bundle holders, show eligible extra single courses with tier-based per-course prices and order total.
- If singleCoursesEnabled is false, display the option as unavailable and explain that single-course purchases are temporarily disabled.

9.2 Local migration and recovery

- Store claimSecret securely until claim succeeds or the pending order expires/rejects.
- Poll with backoff or use push notification to detect approval; avoid fixed-interval high-frequency polling.
- On claim success, atomically migrate local IDs in SQLite and SecureStore, then verify counts/checksums for progress, attempts and download mappings.
- After migration, the app logs in as the registered student and syncs normally.
- After reinstall, a paid student can authenticate against the server and recover entitlements, progress and course downloads. An unpaid local-only account is not recoverable from the server.

9.3 Streaming, downloads and versioning

- Listings are visible immediately from the local catalog, including offline. Listing metadata does not contain protected content bodies.
- Online entitled students can stream without downloading. Offline students can open only downloaded permitted content.
- Downloads resume after interruption and show progress, file size, low-storage warnings and mobile-data warnings.
- Paid downloads are encrypted and tied to the bound device. A signed entitlement with expiry is stored locally and renewed when online.
- Each course has content_version. If server content_version exceeds local_version, show Update available. Download replacement must be atomic so a failed update leaves the prior valid copy intact.
- Delete removes local files and download records; the action returns to Download. Deleting a download does not revoke the purchase entitlement.
- Handle Expo Go versus native development builds explicitly; hardware-backed encryption limitations must be documented and release acceptance tested on supported native builds.

10. Admin Panel Requirements

10.1 Paid-only Students directory

- Only server-created paid/registered students with an assigned MS number appear in Students and student KPIs.

- Pending orders and local-only accounts never appear as student records.
- Student list/profile shows MS number, account status, active entitlements, expiry and permitted operational fields. Sensitive receipt and phone access is audited.

10.2 Receipt queue and approval

- Operations Admin sees pending receipts, order summary, selected courses, course count, bundle min/max validation, expected total, receipt image and status.
- Approval is disabled or rejected server-side if the order is invalid, quote has expired under the configured policy, the amount does not match, or required receipt checks fail.
- On approval, the API performs student creation, counter update, device binding, entitlement creation and order status change atomically.
- Rejection requires a reason and leaves the local account unregistered; the student may correct and resubmit under the pending-order rules.

10.3 Pricing and product settings

- Super Admin can configure batch, semester and full-course products independently.
- Settings include enabled state, min/max count, eligible course scope, pricing mode, fixed price or discount, durationDays, singleCoursesEnabled and extra-course tier rows.
- Course prices are editable and the admin can preview calculated bundle prices for a selected eligible course count.
- Validate non-negative prices, positive duration, minCourses ≥ 1, maxCourses ≥ minCourses, valid discount range, non-overlapping tier ranges and complete tier coverage.
- Price/duration/count-limit changes require a reason, are audited and affect new quotes/orders only, except explicitly approved entitlement adjustments.

11. Domain Functions and Tests

Pure domain functions should be shared through @mastara/domain where practical. API and app tests must cover the same business rule cases.

Function / area	Required behaviour and test coverage
canPurchaseSingleCourse	False without active qualifying bundle; false if course already included/owned; false when single-course sales disabled; true only when all conditions pass.
validateBundleSelection	Reject below minimum, above maximum, duplicate IDs, wrong semester/stream/scope and inactive courses; accept valid boundary counts.
calculateBundlePrice	Calculated mode sums unique course prices then applies configured discount; fixed mode returns fixed price; handles zero/invalid values and currency rounding.
calculateExtraCoursePrice	Applies the correct tier to each extra course ordinal; example 1×100 + 4×50 + 2×40 = 380 ETB for seven extras under the illustrative schedule.
quote/order consistency	Server ignores client total, quote expiry handled, snapshot remains auditable, admin price changes do not alter already accepted order totals.
resolveContentAccess / canOpen	Online entitled content can stream; offline requires download; free access follows the specified precedence; expired/revoked entitlement locks content.
hashClaimSecret / verifyClaimSecret	Valid secret passes, wrong secret fails, missing/expired/unapproved order fails, repeated valid claim is idempotent.
MS counter	Concurrent approved orders get unique sequential IDs; forced transaction rollback does not consume a number; duplicate retry does not create a second student.
Takeover protection	Existing phone/email requires authenticated linking or verified OTP; blind linking rejected.
Receipt endpoint	Rate limits, 2 pending orders per phone, disallowed MIME/content, oversized file and malformed image rejected.
Local migration	Atomic migration preserves progress, attempts and download mappings; retry after partial failure does not duplicate data.
Refund and expiry	Bundle expiry locks bundle-provided access; independent single course remains valid until its own expiry; refund revokes only entitlements linked to refunded order.

12. End-to-End Acceptance Criteria

1. Register a student in airplane mode; verify local UUID exists and no student profile data is sent to the API.
2. Attempt to purchase a single course as a local-only student; the UI is locked and API returns SINGLE_COURSE_BUNDLE_REQUIRED.
3. Select a bundle below minimum and above maximum; both are rejected by the app and API. Boundary counts are accepted.
4. Change a course price or tier setting in Admin; a fresh quote uses the new settings, while an existing valid price snapshot remains unchanged according to quote policy.
5. Submit a valid bundle order with a receipt; student remains local-only and absent from Admin Students while pending.
6. Approve receipt; verify one Student, one MS number, device binding and correct entitlement are created in one transaction.
7. Force a transaction failure; verify no MS number gap. Run concurrent approvals and verify uniqueness and sequence.
8. Claim using the correct secret; receive the same MS number on retries. Wrong secret is rejected.
9. Verify phone/email collision cannot silently link to an existing paid student.
10. Verify local SQLite/SecureStore references migrate atomically from localAccountId to MS number.

11 With active bundle and single-course sales enabled, buy extras; verify tier-by-count calculation and that already-included courses cannot be purchased again.

12 Disable single-course sales; verify new extra orders fail in both app and API, while existing valid entitlements remain usable.

13 Verify online streaming works without download; offline access works only for downloaded content and only while entitlement is valid.

14 Expire the bundle; verify bundle access stops but separately purchased single-course access remains until its own expiry.

15 Refund a bundle or single-course order; only entitlements associated with the refunded order are revoked.

16 Publish a new course version; app shows Update available and a failed download replacement preserves the prior local copy.

17 Run full monorepo tests and integration/e2e suite; no failures and no mock data in production builds.

13. Phase Roadmap

Phase	Scope	Exit gate
P0 — Completed	Baseline assertions and SyncProvider performance/test fixes.	Full monorepo test suite passes with zero failures.
P1 — Contracts/domain/schema	DTOs, pricing/eligibility/access functions, tests, Prisma schema, pricing configuration and course versions.	Unit tests pass; schema migration and rollback tested.
P2 — API and approval/claim	Order hardening, price quotes, receipt validation, transactional MS counter, approval, claim and account collision protection.	Integration tests for price, idempotency, counter rollback, claim security and takeover protection.
P3 — Mobile lifecycle	Local registration, store eligibility, claim polling, secret handling and local migration.	Offline-to-approved-claim flow passes on supported native build.
P4 — Content and downloads	Streaming, resumable download, encryption, expiry and content versioning.	Online stream, offline downloaded access, expiry and update tests pass.
P5 — Admin workflows	Paid-only Students, receipt approval, pricing/count/duration settings.	Role/permission tests and end-to-end approval tests pass.
P6 — End-to-end hardening	Complete purchase-to-learning flow, conflict handling, recovery and regression testing.	All release acceptance gates pass.

14. Decisions and Assumptions to Approve

The following recommendations make the requirements implementable. Product-owner approval should confirm them before Phase 1 schema/domain work is considered final.

Topic	Recommended decision in this document	Why
Payment confirmation	Manual receipt approval at launch; create MS number at approval.	Matches SRS v3.2 and the supplied roadmap; avoids registering unpaid students.
Bundles that unlock singles	Batch, semester and full-course bundles all qualify unless admin disables a type.	Consistent rule; configurable without code changes.
Bundle pricing	Admin-calculated course sum minus configured discount, with optional fixed-price override.	Supports price-by-course-count and business flexibility.
Extra single price by count	Tiered unit prices based on extra-course ordinal within one order.	Predictable preview and auditable total; avoids ambiguous lifetime counting.
Course count limits	Separate min/max settings for batch, semester and full-course products.	Avoids accidentally applying semester rules to other bundle types.
Access start	Starts at receipt approval timestamp.	Clear, auditable and independent of first app use.
Bundle expiry/refund	Expiry affects bundle entitlement; refund revokes entitlements tied to the refunded order only.	Preserves separately purchased course rights unless that purchase is refunded.
Pending price quotes	Quote has an expiry; approved order stores price snapshot.	Prevents stale or manipulated totals while providing a stable payment amount.
Gapless counter	Transactional counter row, not a non-transactional sequence.	Required if strict gapless IDs are a launch requirement.

15. Implementation Notes and Security Warnings

- **Do not persist plaintext passwords.** TLS protects transport; it does not justify storing the submitted password in the pending order. Hash it server-side immediately with bcrypt/Argon2id and clear transient plaintext buffers where feasible. Do not log it.
- **Claim secret:** a hash protects the stored value, but a claim endpoint still needs rate limiting, order-status checks, expiry policy and careful token issuance. Keep the secret only on the device until claim is completed.
- **Gapless IDs:** strict gaplessness requires serializing counter updates and rolling them back with the student-creation transaction. Test rollback and concurrent approvals.
- **Pricing:** the client may display estimates, but only the server quote is authoritative. Use integer minor units or decimal arithmetic, never floating-point money.
- **Local encryption:** downloaded paid files and local entitlements must be protected by device-bound keys and signed expiry. Encryption alone cannot prevent every form of copying on a compromised/rooted device; treat this as risk reduction, not absolute anti-sharing.
- **Operational safety:** rotate any secret ever committed to Git, keep receipts private, remove EXIF, limit access, define retention, and audit receipt views and pricing changes.

16. Final Requirement Statement

Mastara shall calculate bundle and extra-course prices from server-controlled course counts and pricing rules, enforce configurable bundle minimum/maximum sizes, and permit extra single-course purchases only for registered students with an active qualifying bundle when single-course sales are enabled. The server shall validate every order, compute and snapshot the amount due, and create a student/MS number and entitlements only after the first qualifying bundle receipt is approved. Access shall follow the entitlement duration and offline-first rules defined in this specification.

Source basis: Mastara SRS v3.2 (25 October 2026); Mastara SRS — Update Sections (after v3.2) (27 October 2026); and the implementation refinements and phase roadmap supplied by the project owner on 28 October 2026. The pricing formulas, illustrative tier example, and explicit resolutions are recommendations added to remove ambiguity; they are not presented as previously approved values.